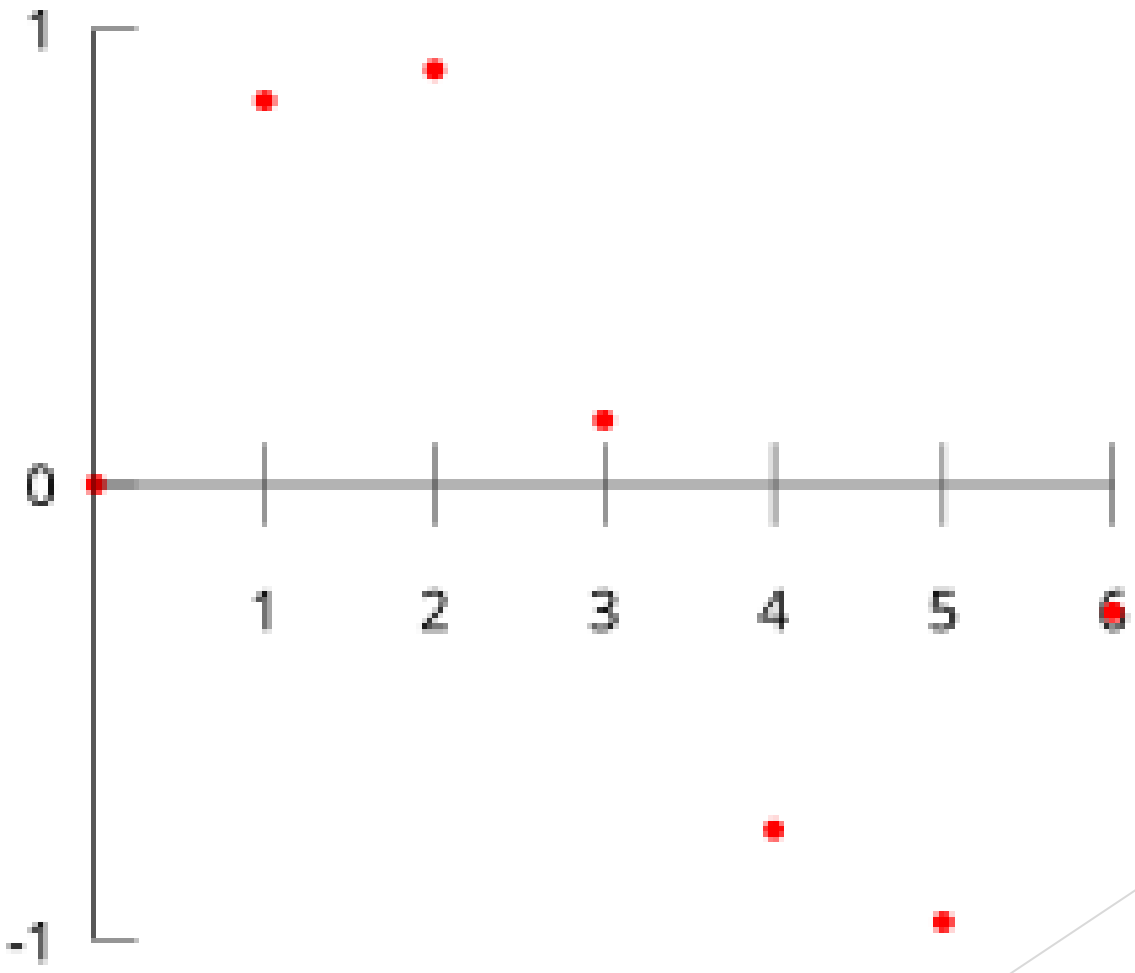
Interpolation

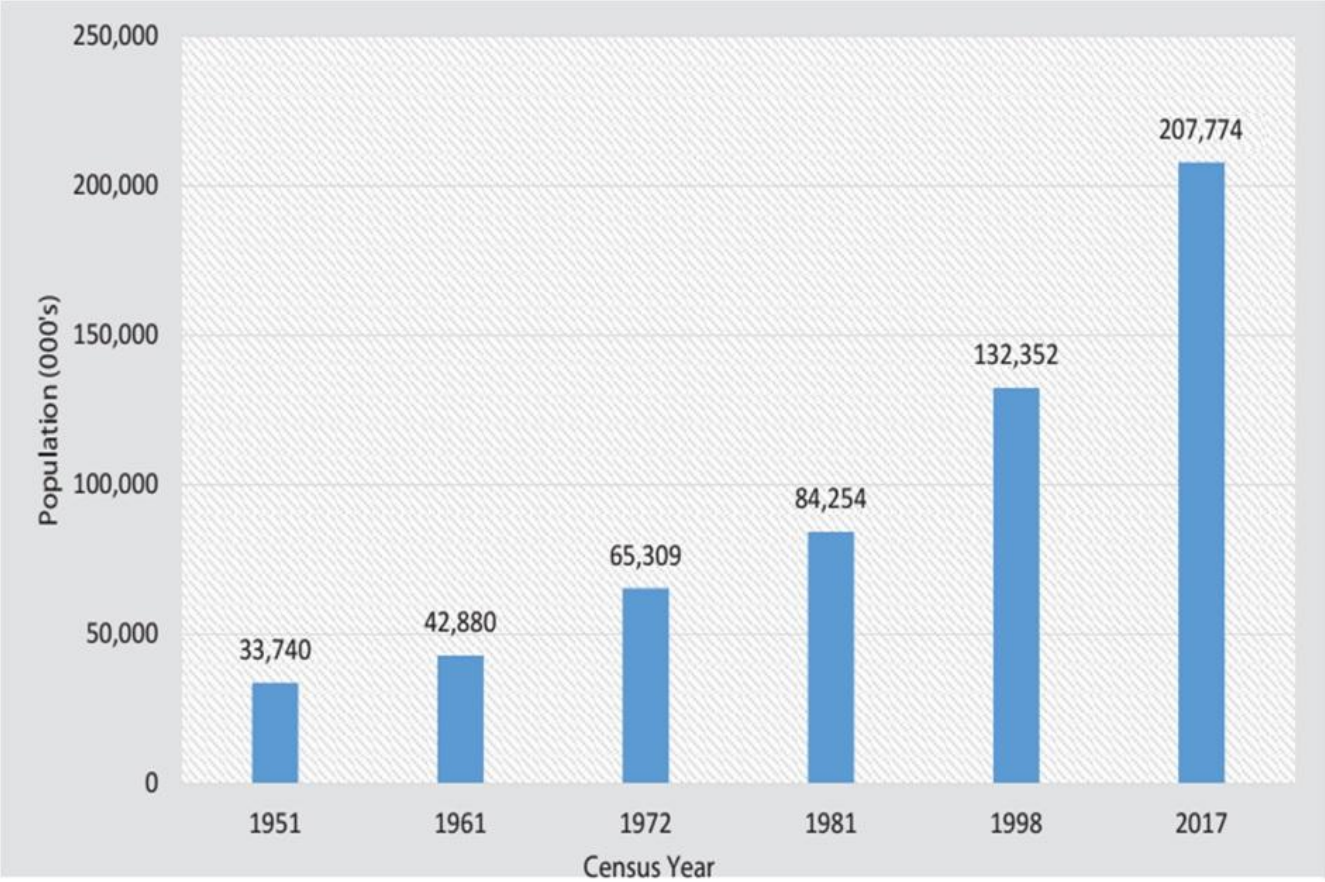
Introduction

Interpolation is a method of estimating new, unknown data points that lie between known, discrete data points

Interpolation



Interpolation



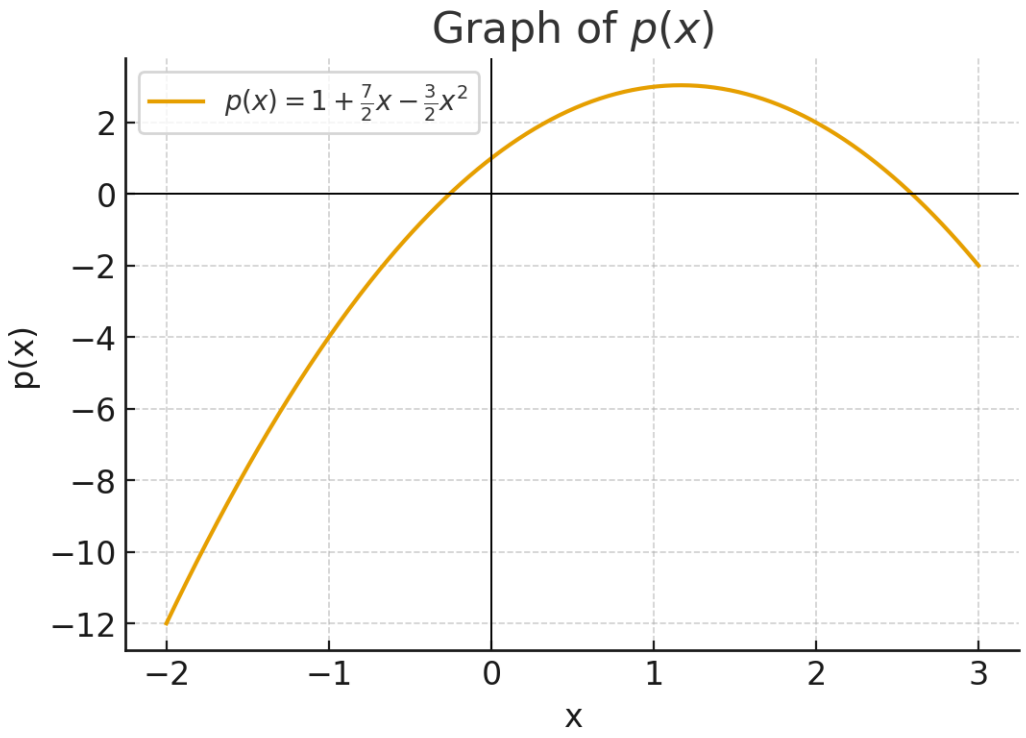
Introduction

Interpolation is the process of deriving a simple function from a set of discrete data points so that the function passes through all the given data points

Use **3 points**:

$$(0, 1), \quad (1, 3), \quad (2, 2).$$

We expect a quadratic $p(x) = a + bx + cx^2$.



Interpolation

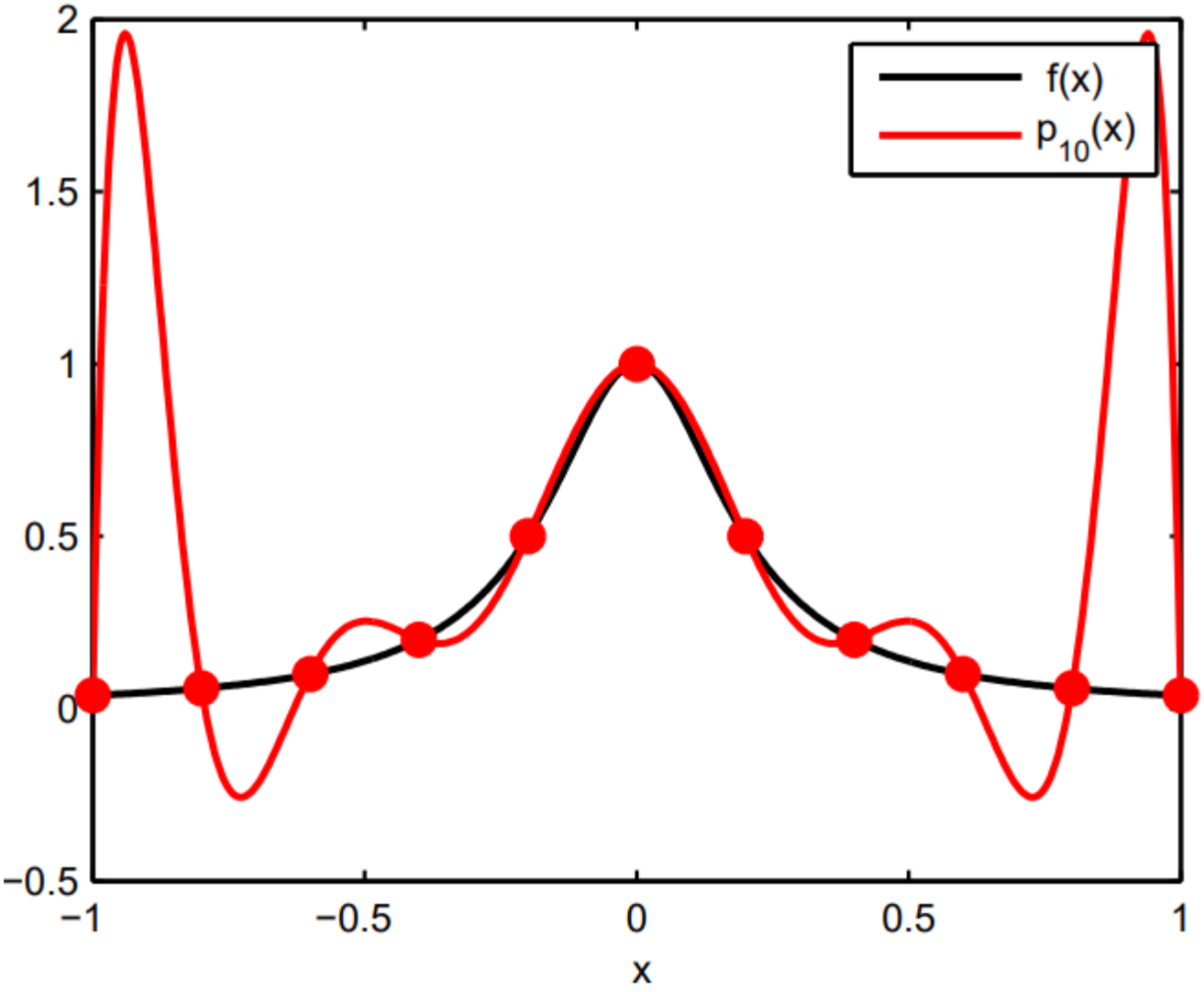
Given n+1 data points the direct method assumes the following polynomial:

$$y = f(x) = a_0 + a_1 x + a_2 x^2 + \ldots + a_n x^n$$

With n+1 values for x and the n+1 corresponding values for y we can solve for $a_0, a_1, \ldots, a_n$ by solving the n+1 simultaneous linear equations, which is known as the direct method.

Introduction

Interpolation is the problem of fitting a smooth curve through a given set of points, generally as the graph of a function.


f(x)
p10(x)
2
1.5
1
0.5
0
−0.5
−1
−0.5
0
0.5
1
x

Vandermonde matrix

Interpolation using the Vandermonde matrix

The most basic procedure to determine the coefficients $a_0, a_1, \ldots, a_n$ of a polynomial

$$P_n(x) = a_0 + a_1 x + a_2 x^2 + \cdots + a_n x^n$$

such that it interpolates the $n+1$ points

$$(x_0, y_0), (x_1, y_1), \ldots, (x_n, y_n)$$

is to write a linear system of equations as follows:

$$\begin{aligned}
P_n(x_0) = y_0 &\Rightarrow a_o + a_1 x_0 + a_2 x_0^2 + \cdots + a_{n-1} x_0^{n-1} + a_n x_0^n = y_0 \\
P_n(x_1) = y_1 &\Rightarrow a_o + a_1 x_1 + a_2 x_1^2 + \cdots + a_{n-1} x_1^{n-1} + a_n x_1^n = y_1 \\
\vdots &\Rightarrow \vdots \\
P_n(x_n) = y_n &\Rightarrow a_o + a_1 x_n + a_2 x_n^2 + \cdots + a_{n-1} x_n^{n-1} + a_n x_n^n = y_n
\end{aligned}$$

or, in matrix form:

$$\underbrace{\begin{bmatrix} 1 & x_0 & x_0^2 & \ldots & x_0^{n-1} & x_0^n \\ 1 & x_1 & x_1^2 & \ldots & x_1^{n-1} & x_1^n \\ \vdots & \vdots & \vdots & \vdots & \vdots & \\ 1 & x_{n-1} & x_{n-1}^2 & \ldots & x_{n-1}^{n-1} & x_{n-1}^n \\ 1 & x_n & x_n^2 & \ldots & x_n^{n-1} & x_n^n \end{bmatrix}}_{\mathbf{V}} \underbrace{\begin{bmatrix} a_0 \\ a_1 \\ \vdots \\ a_{n-1} \\ a_n \end{bmatrix}}_{\vec{a}} = \underbrace{\begin{bmatrix} y_0 \\ y_1 \\ \vdots \\ y_{n-1} \\ y_n \end{bmatrix}}_{\vec{b}}$$

Vandermonde matrix

Use the expression of p_N into the interpolating equations $y_j = p_N(x_j)$:

$$\sum_{n=0}^{N} a_n x_j^n = y_j, \qquad j = 0, \ldots, N.$$

In these $N+1$ equations indexed by j, the unknowns are the coefficients $a_0, \ldots, a_N$. We are in presence of a linear system

$$V\mathbf{a} = \mathbf{y}, \qquad \Leftrightarrow \qquad \sum_{n=0}^{N} V_{jn} a_n = y_j,$$

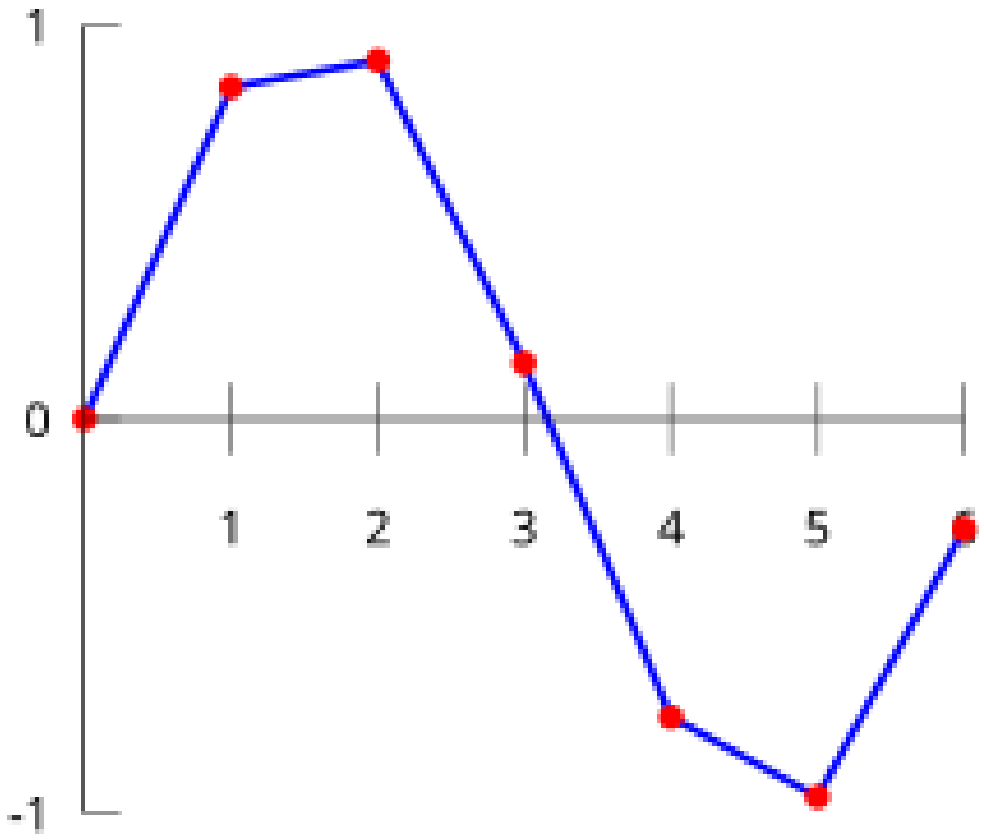
with V the so-called Vandermonde matrix, $V_{jn} = x_j^n$, i.e.,

$$V = \begin{pmatrix} 1 & x_0 & \cdots & x_0^N \\ 1 & x_1 & \cdots & x_1^N \\ \vdots & \vdots & & \vdots \\ 1 & x_N & \cdots & x_N^N \end{pmatrix}.$$

Types of Interpolation

- Linear interpolation
- Nearest Neighbor Interpolation
- Polynomial interpolation
- Spline interpolation

Linear interpolation



Linear interpolation

$$y = y_a + (y_b - y_a)\,\frac{x - x_a}{x_b - x_a} \text{ at the point } (x, y)$$

$$\frac{y - y_a}{y_b - y_a} = \frac{x - x_a}{x_b - x_a}$$

$$\frac{y - y_a}{x - x_a} = \frac{y_b - y_a}{x_b - x_a}$$

Slop of these two points will be the same as the slop of the polynomial

Nearest Neighbours Interpolation

Nearest neighbours interpolation is the simplest interpolation method.

$$(1, 2), (3, 5), (6, 1)$$

Polynomial interpolation

Given $N+1$ points $x_j \in \mathbb{R}$, $0 \leq j \leq N$, and sample values $y_j = f(x_j)$ of a function at these points, the polynomial interpolation problem consists in finding a polynomial $p_N(x)$ of degree N which reproduces those values:

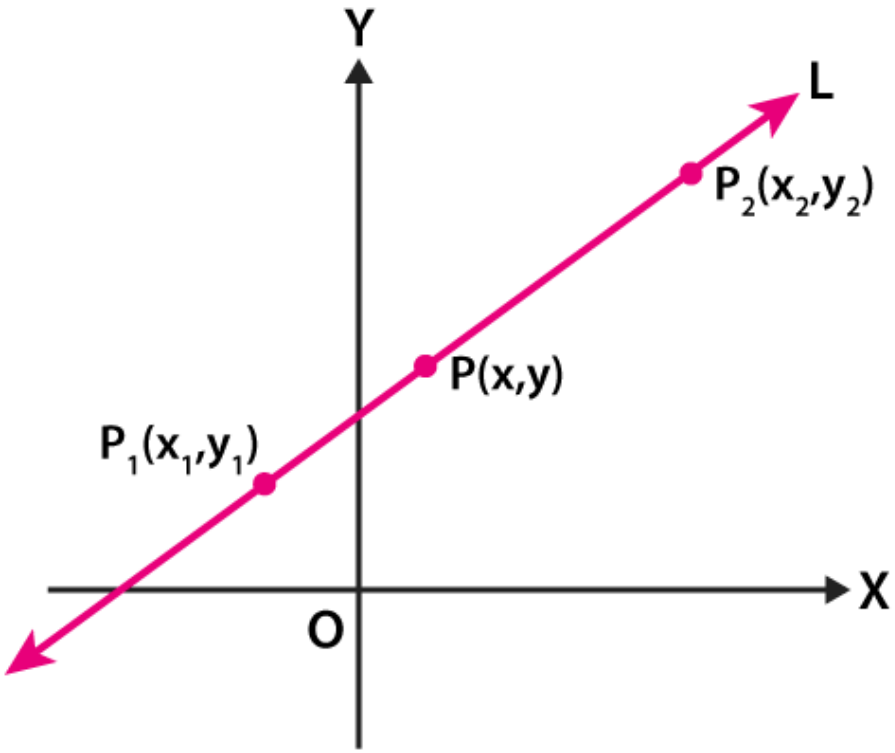
$$y_j = p_N(x_j), \qquad j = 0, \ldots, N.$$

Polynomial interpolation

Given a set of data, polynomial interpolation is a method of finding a polynomial function that fits a set of data points exactly. Though there are several methods for finding this polynomial

Quadratic Interpolation

Quadratic Interpolation



Quadratic Interpolation

$$y - y_1 = m(x - x_1) \quad \text{and} \quad m = \frac{y_1 - y_0}{x_1 - x_0}$$

$$y = \frac{y_1 - y_0}{x_1 - x_0}(x - x_1) + y_1$$

$$y = \frac{x - x_1}{x_1 - x_0}(y_1 - y_0) + y_1$$

$$y = \frac{x - x_1}{x_1 - x_0}y_1 - \frac{x - x_1}{x_1 - x_0}y_0 + y_1$$

$$y = \frac{x - x_1}{x_1 - x_0}y_1 - \frac{x - x_1}{x_1 - x_0}y_0 + \frac{x_1 - x_0}{x_1 - x_0}y_1$$

$$y = -\frac{x - x_1}{x_1 - x_0}y_0 + \frac{x - x_0}{x_1 - x_0}y_1$$

$$y = \frac{x - x_1}{x_0 - x_1}y_0 + \frac{x - x_0}{x_1 - x_0}y_1.$$

Quadratic Interpolation

$$y = \frac{x - x_1}{x_0 - x_1} y_0 + \frac{x - x_0}{x_1 - x_0} y_1.$$

For three data points $(x_0, y_0), (x_1, y_1), (x_2, y_2)$, we can derive a polynomial that behaves similarly:

$$y = \frac{(x - x_1)(x - x_2)}{(x_0 - x_1)(x_0 - x_2)} y_0 + \frac{(x - x_0)(x - x_2)}{(x_1 - x_0)(x_1 - x_2)} y_1 + \frac{(x - x_0)(x - x_1)}{(x_2 - x_0)(x_2 - x_1)} y_2$$

Lagrange Interpolation

Lagrange Polynomial (General Form)

Given $n+1$ data points $(x_0, y_0), (x_1, y_1), \ldots, (x_n, y_n)$ with $x_0 < x_1 < \cdots < x_n$, the Lagrange polynomial is the nth degree polynomial passing through each of these points and written as:

$$y = \frac{(x-x_1)(x-x_2)\cdots(x-x_n)}{(x_0-x_1)(x_0-x_2)\cdots(x_0-x_n)}y_0 + \frac{(x-x_0)(x-x_2)\cdots(x-x_n)}{(x_1-x_0)(x_1-x_2)\cdots(x_1-x_n)}y_1 + \cdots + \frac{(x-x_0)(x-x_1)\cdots(x-x_{n-1})}{(x_n-x_0)(x_n-x_1)\cdots(x_n-x_{n-1})}y_n$$

Equivalently, we can use product notation:

$$y = \sum_{i=0}^{n} y_i \left(\prod_{\substack{j=0 \\ j \neq i}}^{n} \frac{x - x_j}{x_i - x_j} \right)$$

```
import numpy as np
import matplotlib.pyplot as plot

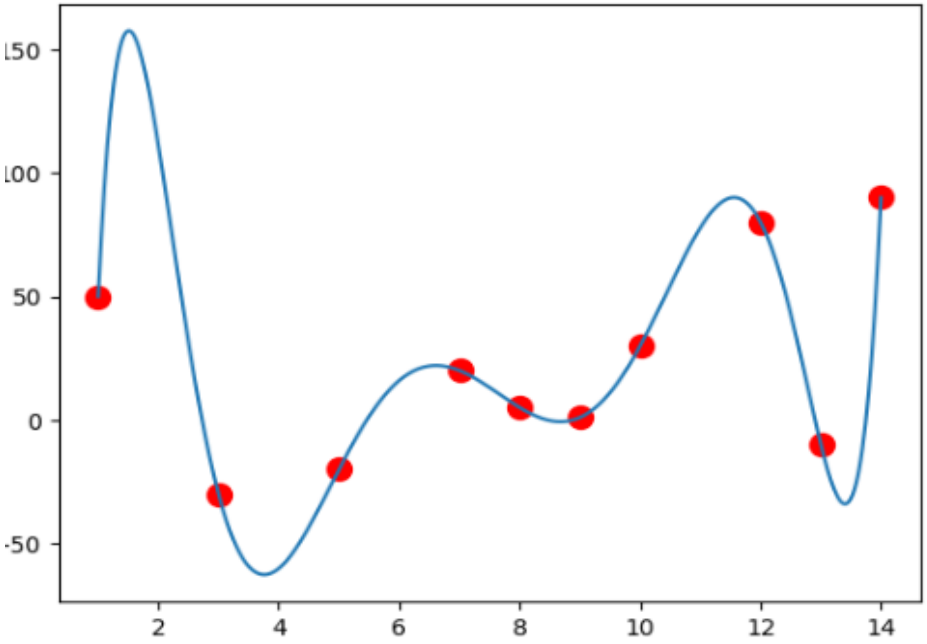
x=[1,3,5,7,8,9,10,12,13, 14]
y=[50,-30,-20,20,5,1,30,80,-10, 90]

n=len(x)-1
prange = np.linspace(min(x),max(x),500)

plot.plot(x,y,marker='o', color='r', ls='', markersize=10)

def f(o):
  sum = 0
  for i in range(n+1):
    prod = y[i]
    for j in range(n+1):
      if i!= j:
        prod=prod*(o-x[j])/(x[i]-x[j])
    sum = sum + prod
  return sum

plot.plot(prange,f(prange))
plot.show()
```



Home Task: Lagrange Interpolation (5-Marks)

- Implement Lagrange interpolation manually in Python.
- Identify and use the SciPy library function that provides Lagrange interpolation.
- Compare both on a 3rd-order any polynomial using 5 data points.
- Plot results and discuss differences (if any).